



İSTANBUL TİCARET
ÜNİVERSİTESİ

ENM404 PRODUCTION SCHEDULING

Job-shop Scheduling Optimization Using Tabu Search Algorithm

Prepared by:

200026310 Beyza Ay
200026411 Ebrar Öztürk
200026506 İrem Ubuz

Supervisor: Prof. Dr. Mehmet Emin Aydın

May 2026

What Is the Job-Shop Scheduling Problem (JSSP)?

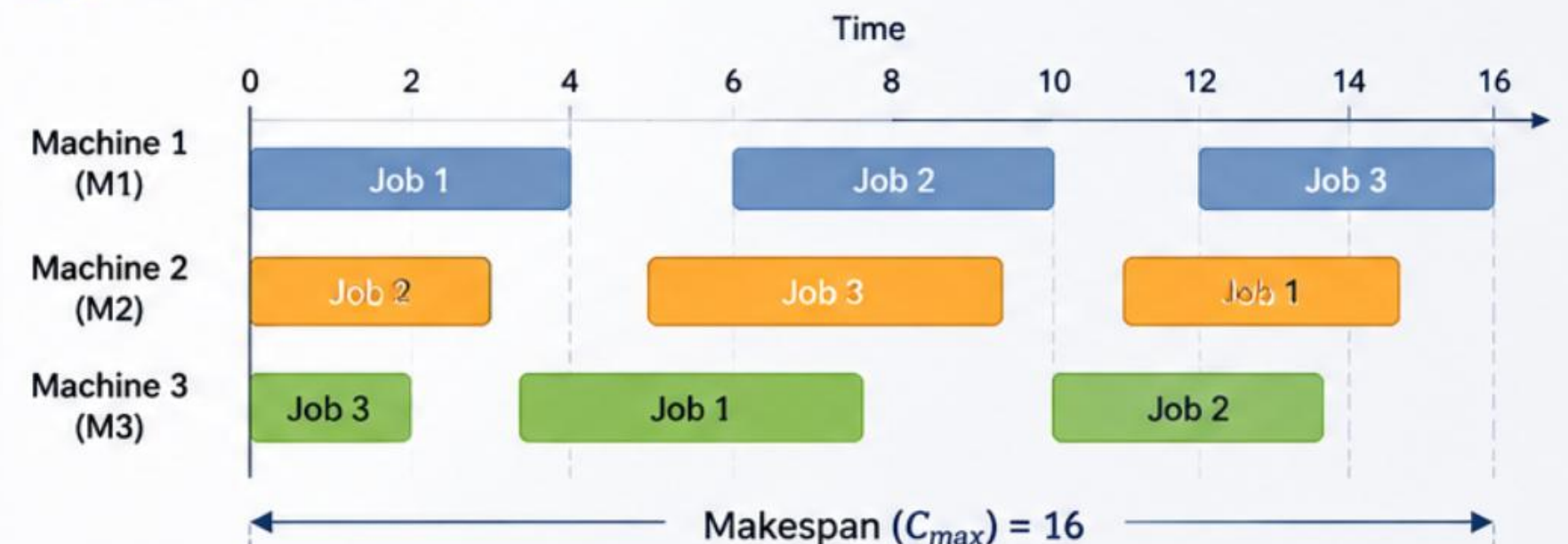
- Multiple jobs consist of ordered operations
- Each operation requires a specific machine
- Machines process one operation at a time
- Precedence constraints must be satisfied
- Minimize total completion time (Makespan)

$$C_{max} = \max(C_j)$$

EXAMPLE JOB SEQUENCES



SAMPLE SCHEDULE (GANTT CHART)



JSSP is an **NP-Hard** combinatorial optimization problem.

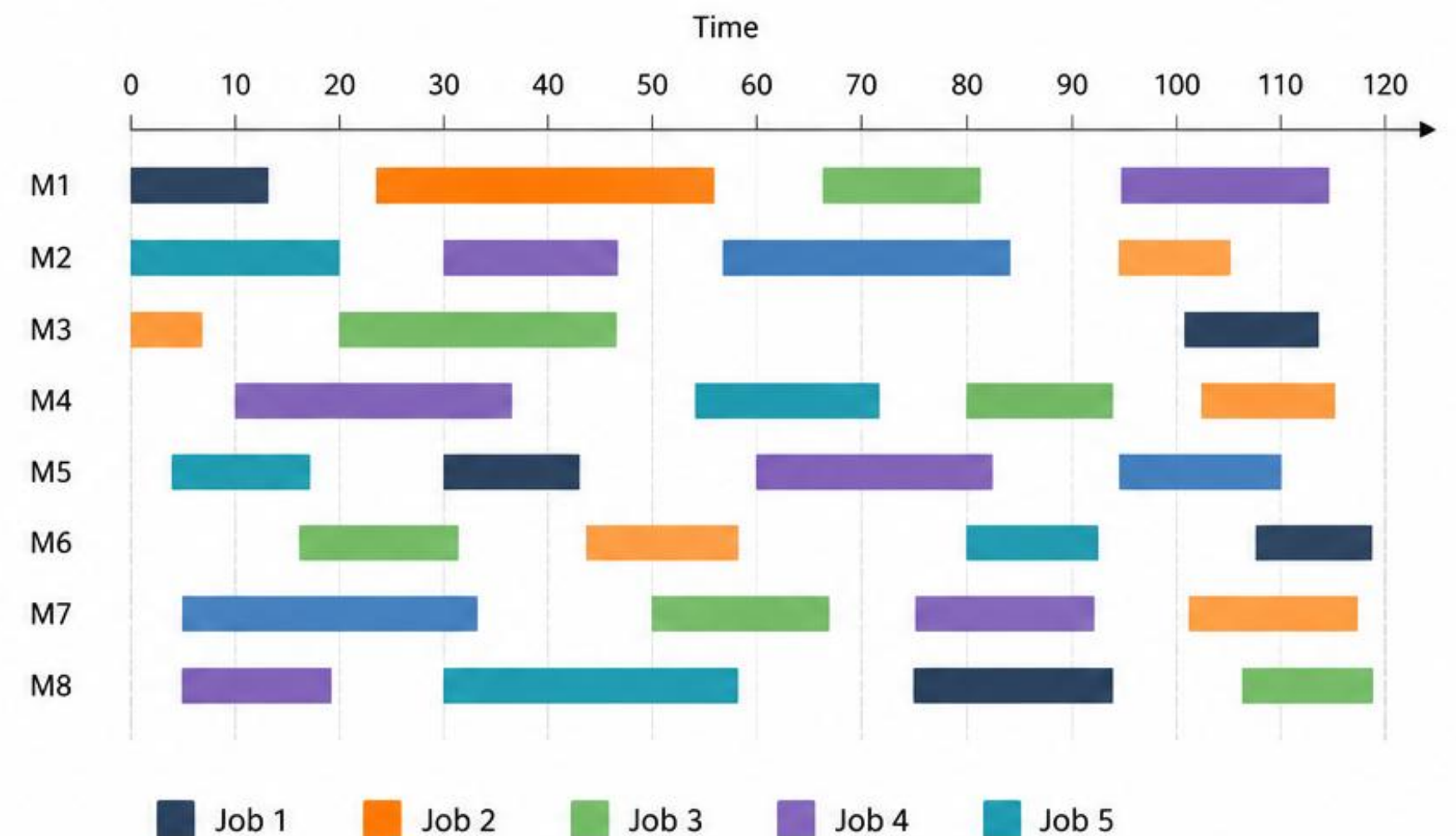
Why Is It Important?

- Reduces machine idle times
- Improves production efficiency
- Increases resource utilization
- Minimizes total makespan
- Supports effective production planning

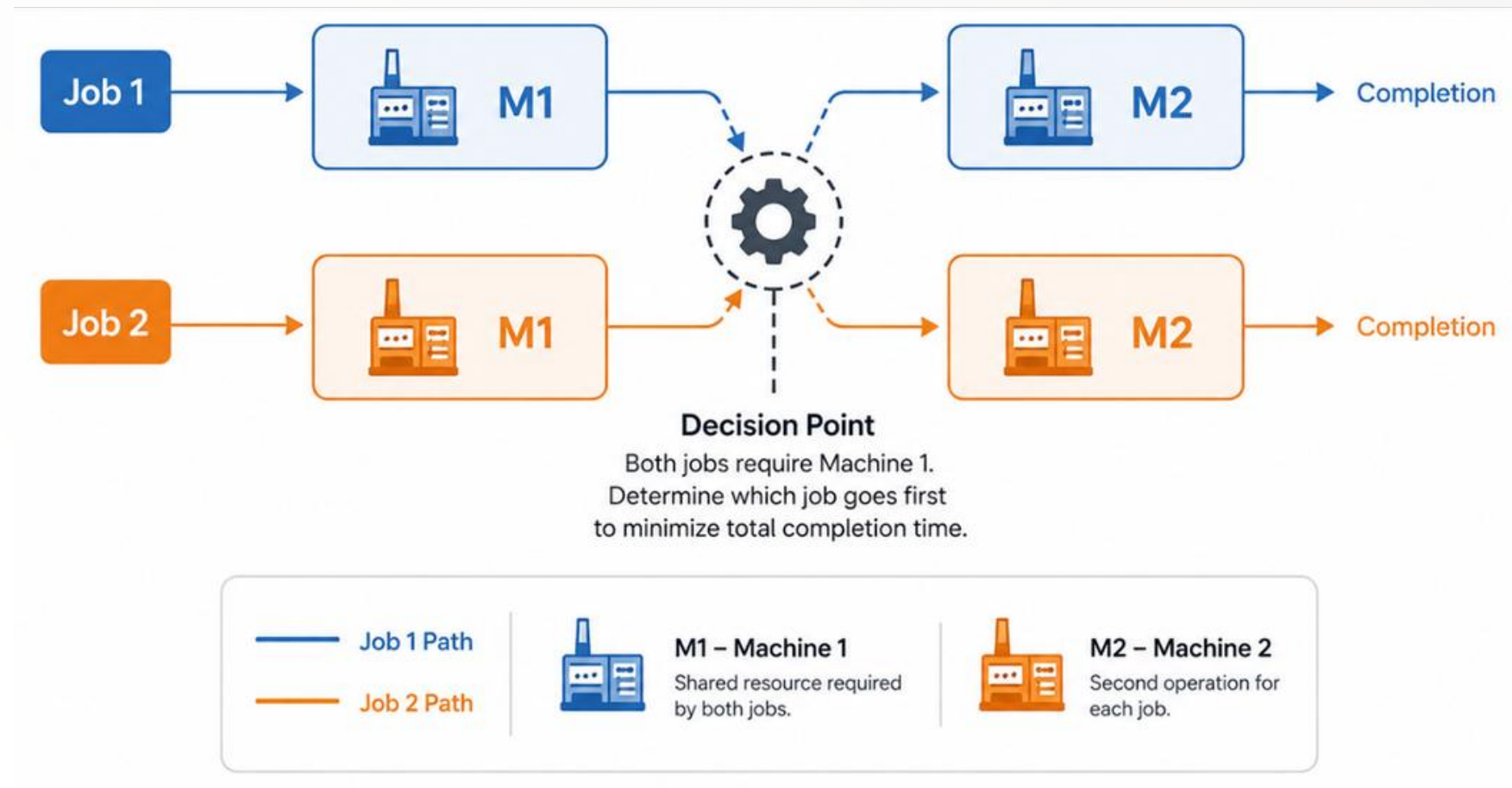


Main Objective: Makespan Minimization

- **Objective:** minimize the makespan (C_{max})
- C_{max} represents the completion time of the final operation in the schedule
- Lower makespan leads to higher scheduling efficiency
- Reducing makespan shortens the overall production duration



Makespan (C_{max}) = Total completion time of the schedule.



Problem Definition

Each job consists of multiple operations

Operations follow a predefined sequence

Each operation requires a specific machine

Each machine can process only one operation at a time

Objective: minimize makespan (C_{max})



Sets

- J : Set of jobs
- M : Set of machines
- O_j : Operations of j
- O_{jr} : The r th operation of job j



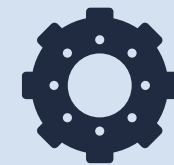
Parameters

- p_{jr} : Processing time
- m_{jr} : required machine
- n : Number of jobs
- k : Number of machines
- C_{max} : makespan



Decision Variables

- S_{jr} : Starting time of operation
- C_{jr} : Completion time
- C_{max} : final completion time
- x_{ih} : Sequencing variable



Objective Function

- Minimize total completion time
- Reduce machine idle times
- Improve production efficiency

$\min C_{max}$

MATHEMATICAL CONSTRAINTS

Completion Constraint

Ensures that completion time equals start time plus processing time.

$$C_{jr} = S_{jr} + p_{jr}$$

Makespan Constraint

Makespan must be greater than all completion times.

$$C_{\max} \geq C_{jr}$$

Precedence Constraint

Operations must follow the predefined job order.

$$S_{j,r+1} \geq C_{jr}$$

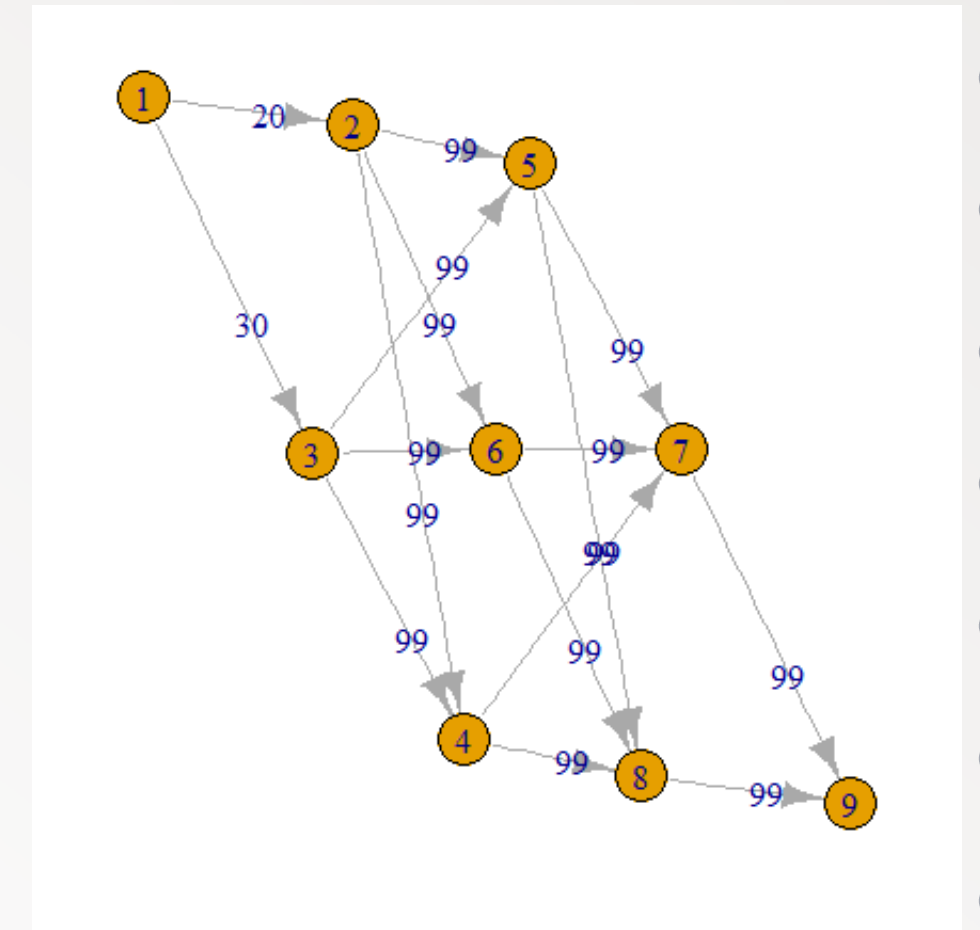
Machine Capacity Constraint

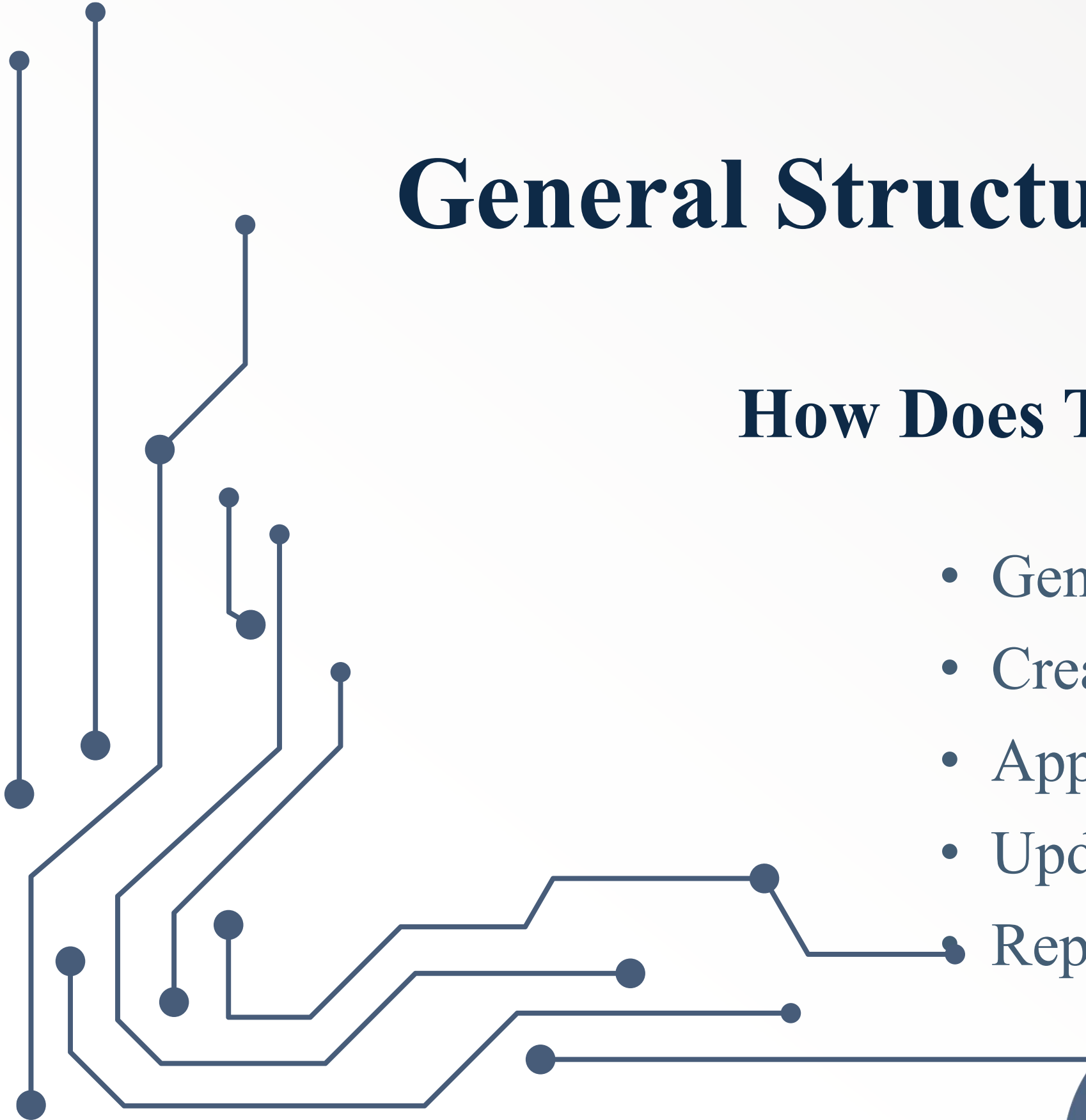
Machines can process only one operation at a time.

$$S_a \geq C_b \text{ or } S_b \geq C_a$$

Why Tabu Search?

1. Exact methods become computationally expensive
2. Metaheuristics provide near-optimal solutions
3. Tabu Search uses memory structures
4. Prevents repeated search patterns
5. Effective for large-scale JSSP problems





General Structure of Tabu Search

How Does Tabu Search Work?

- Generate Initial Solution
 - Create Neighbor Solutions
 - Apply Tabu Restrictions
 - Update Best Solution
 - Repeat Until Stopping Criterion
- 

SOLUTION REPRESENTATION

Operation-Based Representation

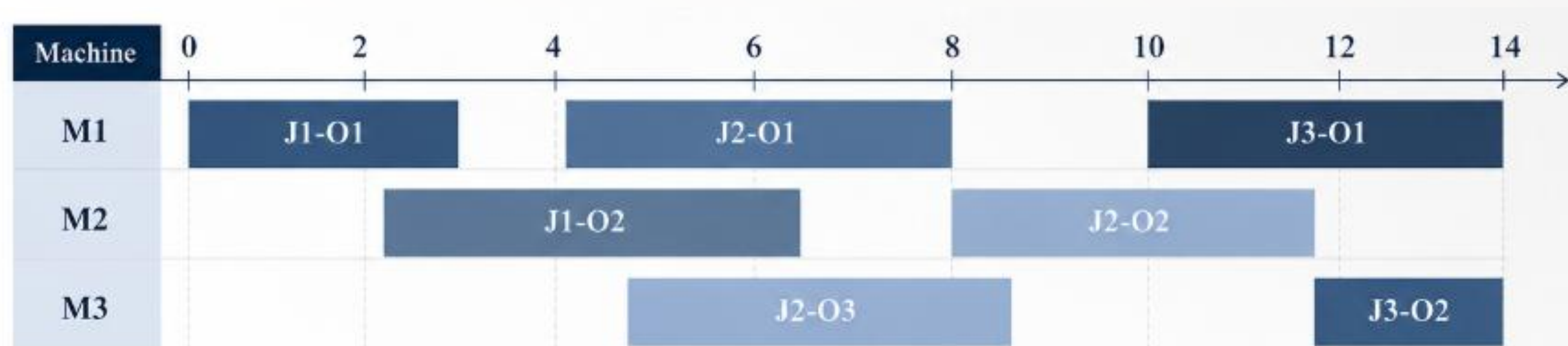
J1-O1 → J2-O1 → J1-O2 → J3-O1

- Solutions are represented as operation sequences
- Job precedence constraints are preserved
- The sequence determines machine order

INITIAL SOLUTION GENERATION

Initial Feasible Schedule

- A feasible initial schedule is generated
- Operations are assigned while preserving precedence constraints
- The search process starts from this initial solution



NEIGHBOURHOOD STRUCTURE

Swap-Based Neighborhood Search

Current Solution

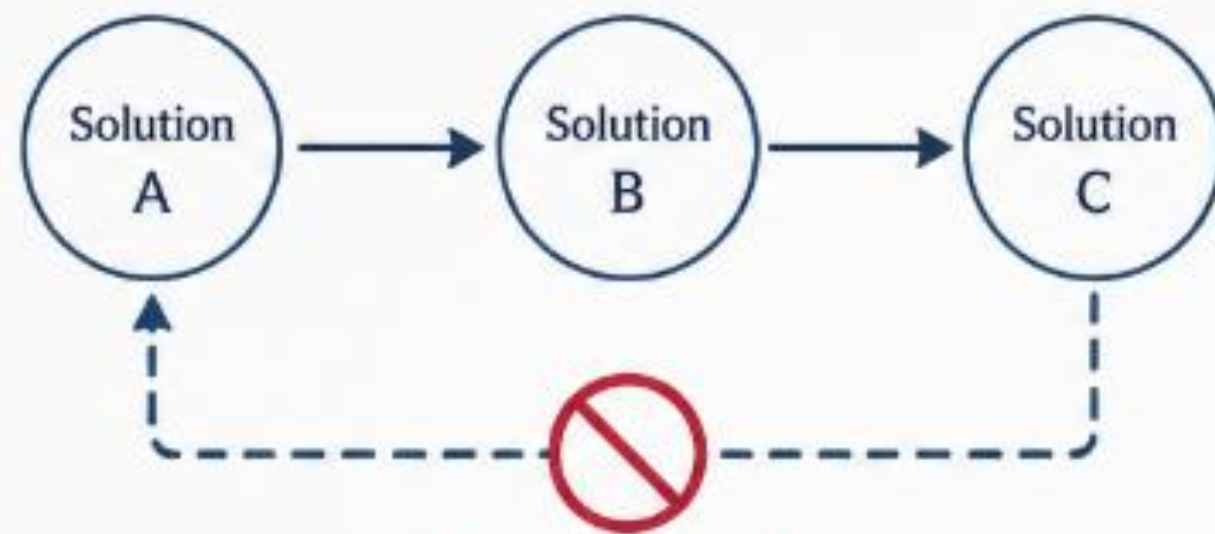
$J1 \rightarrow J2 \rightarrow J3 \rightarrow J4$

Neighbor Solution

$J1 \rightarrow J3 \rightarrow J2 \rightarrow J4$

- Neighbor solutions are generated by swap moves
- Makespan is recalculated after each move

TABU LIST MECHANISM



Forbidden (Tabu) Move

Tabu List (Example)

Move (Swap)	Tabu Tenure
J2 ↔ J3	5
J1 ↔ J4	3

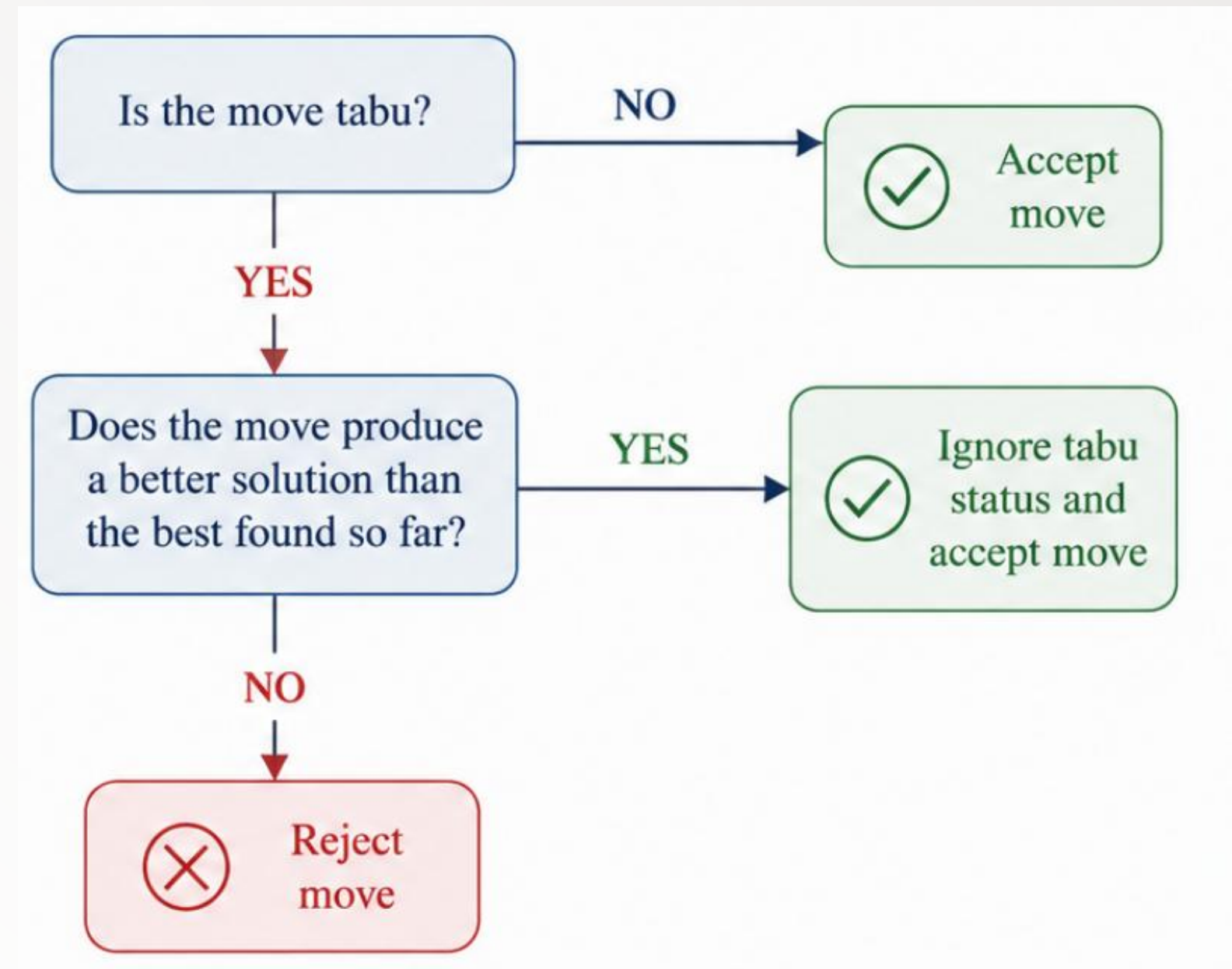
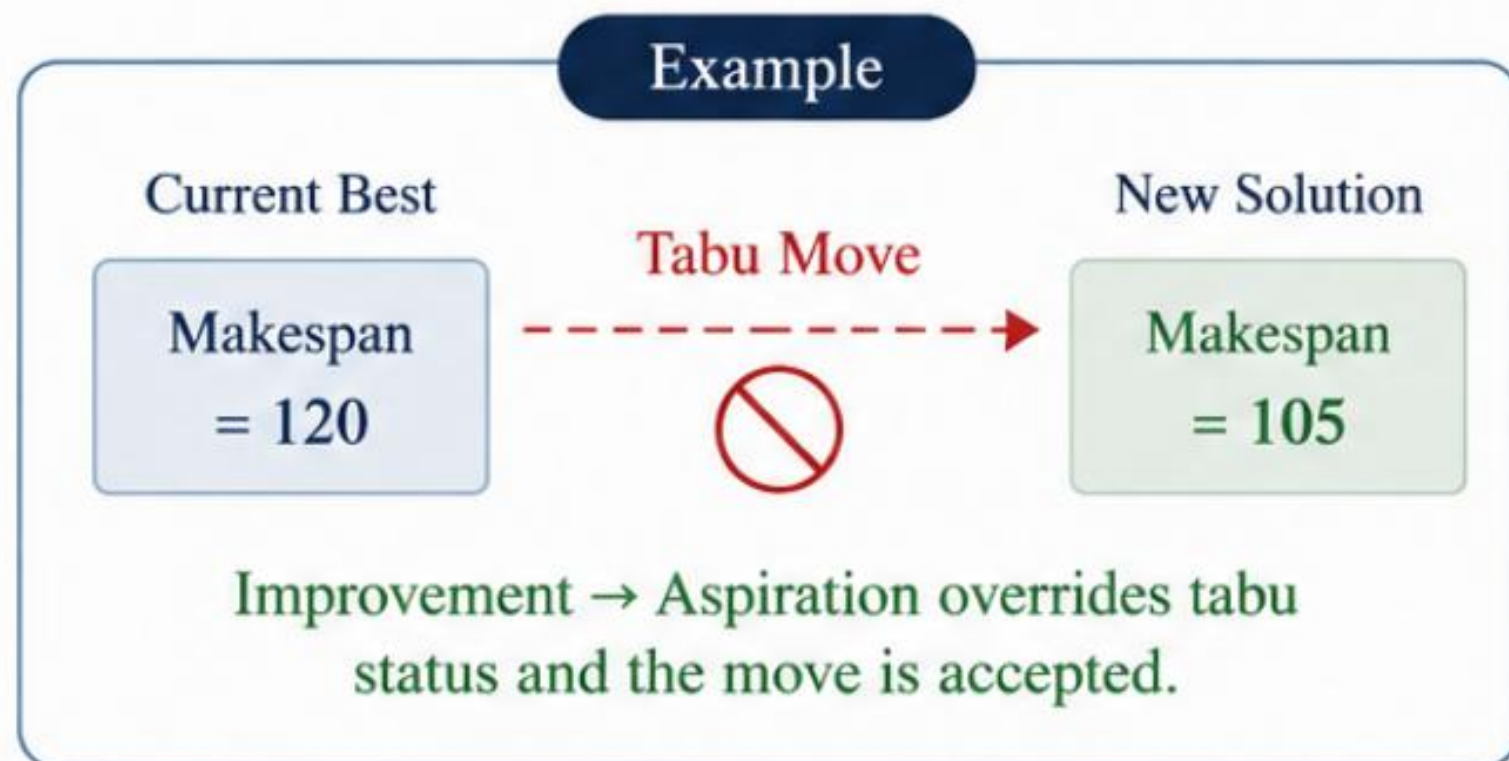
Preventing Repeated Solutions

1. Recently visited moves are stored temporarily
2. Prevents cycling between solutions
3. Helps the algorithm escape local optimum

Aspiration Criterion

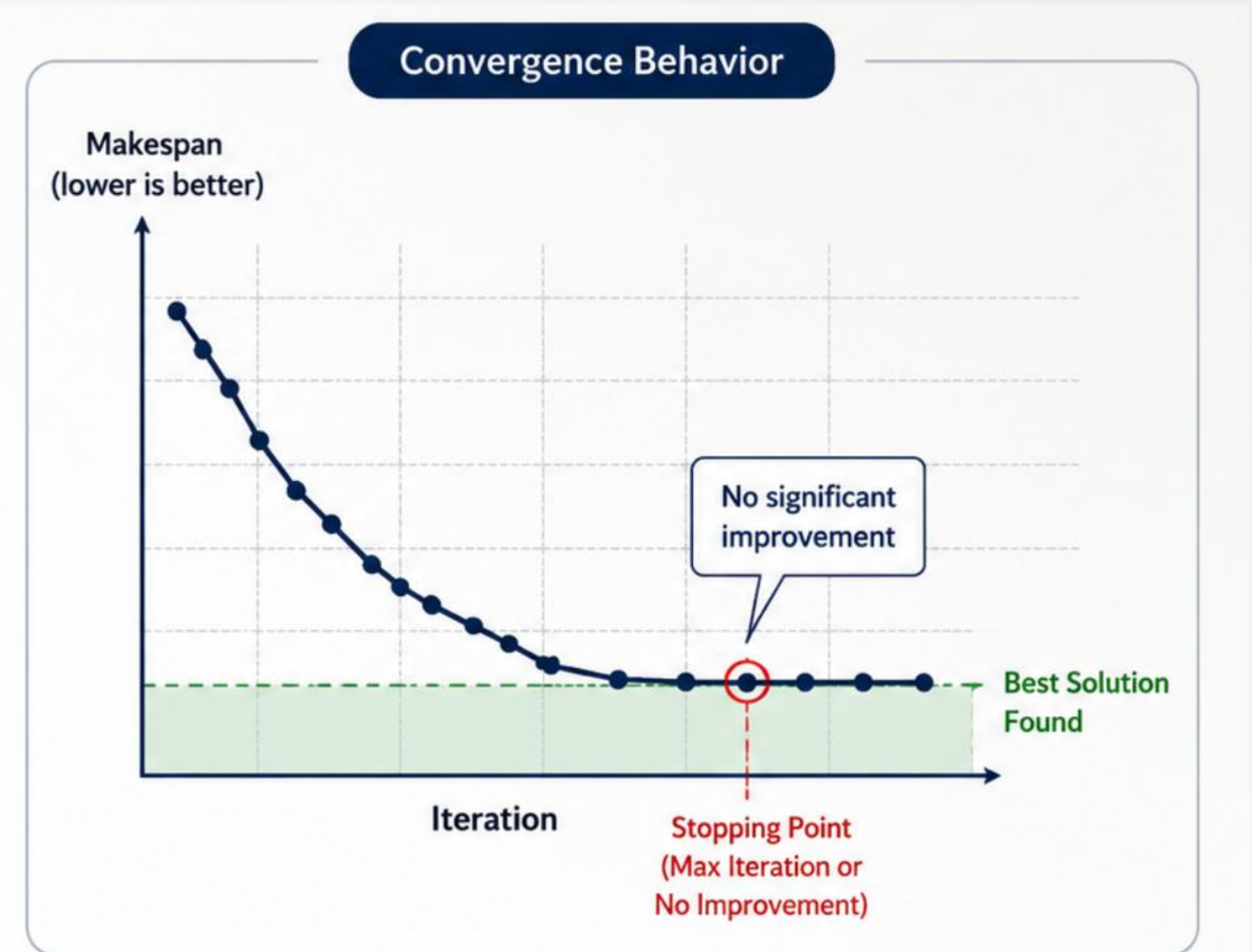
Allowing Better Solutions

- Some tabu moves may produce better schedules
- Tabu status is ignored if the solution improves the best result
- Balances intensification and diversification



When Does the Algorithm Stop?

- Maximum iteration limit reached
- No further improvement observed
- Computational limit exceeded
- In this study, iteration-based stopping was used



Experimental Setup



WHY IS IT EFFECTIVE?

- ✓ Produces near-optimal solutions
- ✓ Escapes local optima efficiently
- ✓ Effective for large-scale JSSP problems



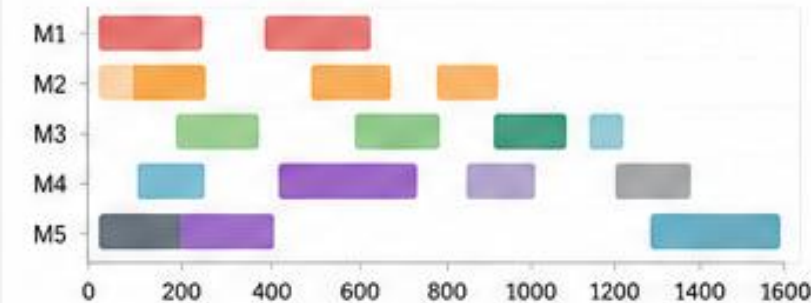
PRACTICAL BENEFITS

- ✓ Improves machine utilization
- ✓ Reduces production completion time



IMPACT ON SCHEDULING PERFORMANCE

BEFORE (Without Tabu Search)



Makespan

1500



Utilization

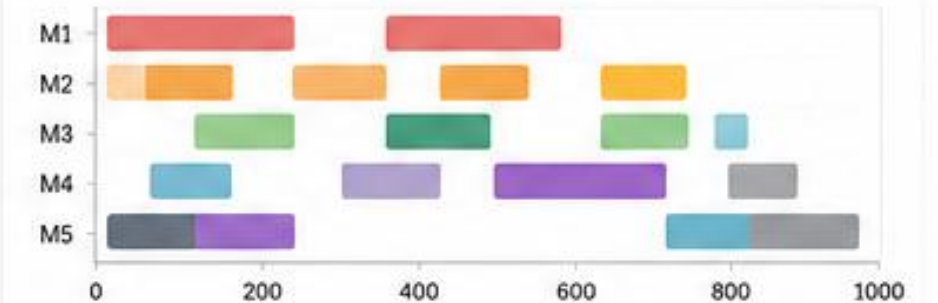
72%



Completion Time

High

AFTER (With Tabu Search)



Makespan

980

↓ 35%



Utilization

89%

↑ 17%



Completion Time

Lower

↓ Significantly

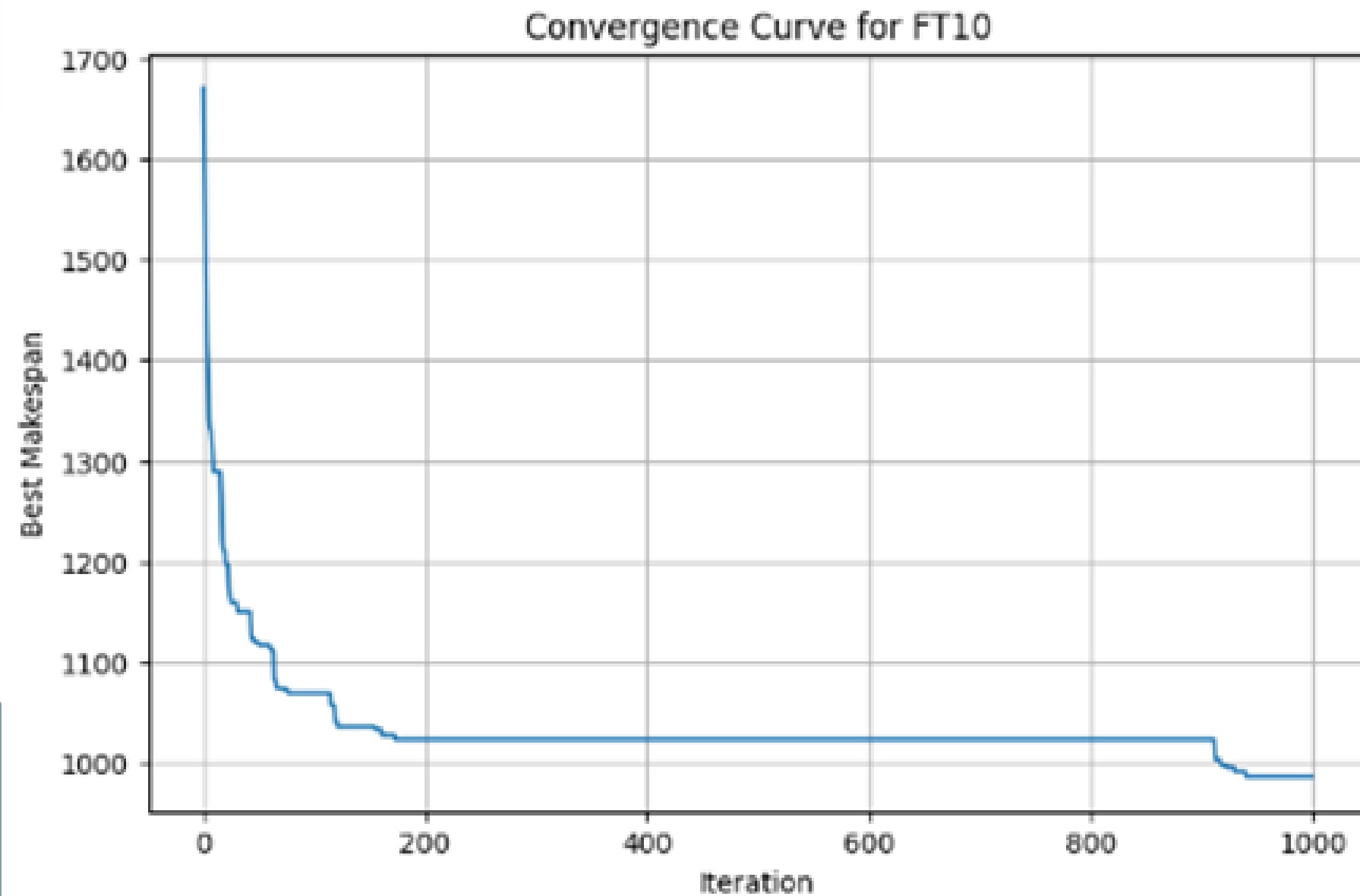
Benchmark Instances

Instance	Size (Jobs × Machines)	Difficulty Level
FT10	10 jobs × 10 machines	Easy ✓
FT20	20 jobs × 5 machines	Moderate −
LA30	20 jobs × 10 machines	Difficult !
LA35	30 jobs × 10 machines	Difficult !
LA40	15 jobs × 15 machines	Very Difficult !!



- Standard benchmark sets from the literature
- Used to evaluate algorithm performance

Convergence Behavior



Search Improvement Over Iterations

- Makespan decreases throughout the search
- Rapid improvement occurs in early iterations
- Stable convergence behavior is achieved

Computational Results

Each benchmark instance was solved 10 times

Performance was evaluated using makespan, gap, and computational time

Results were compared with best-known benchmark values

Instance	Best Known Makespan	Best Found Makespan	Gap (%)	Avg. Time (s)
FT10	930	986	6.02	4.02
FT20	1165	1216	4.38	3.85
LA30	1355	1355	0.00	7.82
LA35	1888	1888	0.00	11.52
LA40	1222	1297	6.14	8.72



RESULT SUMMARY

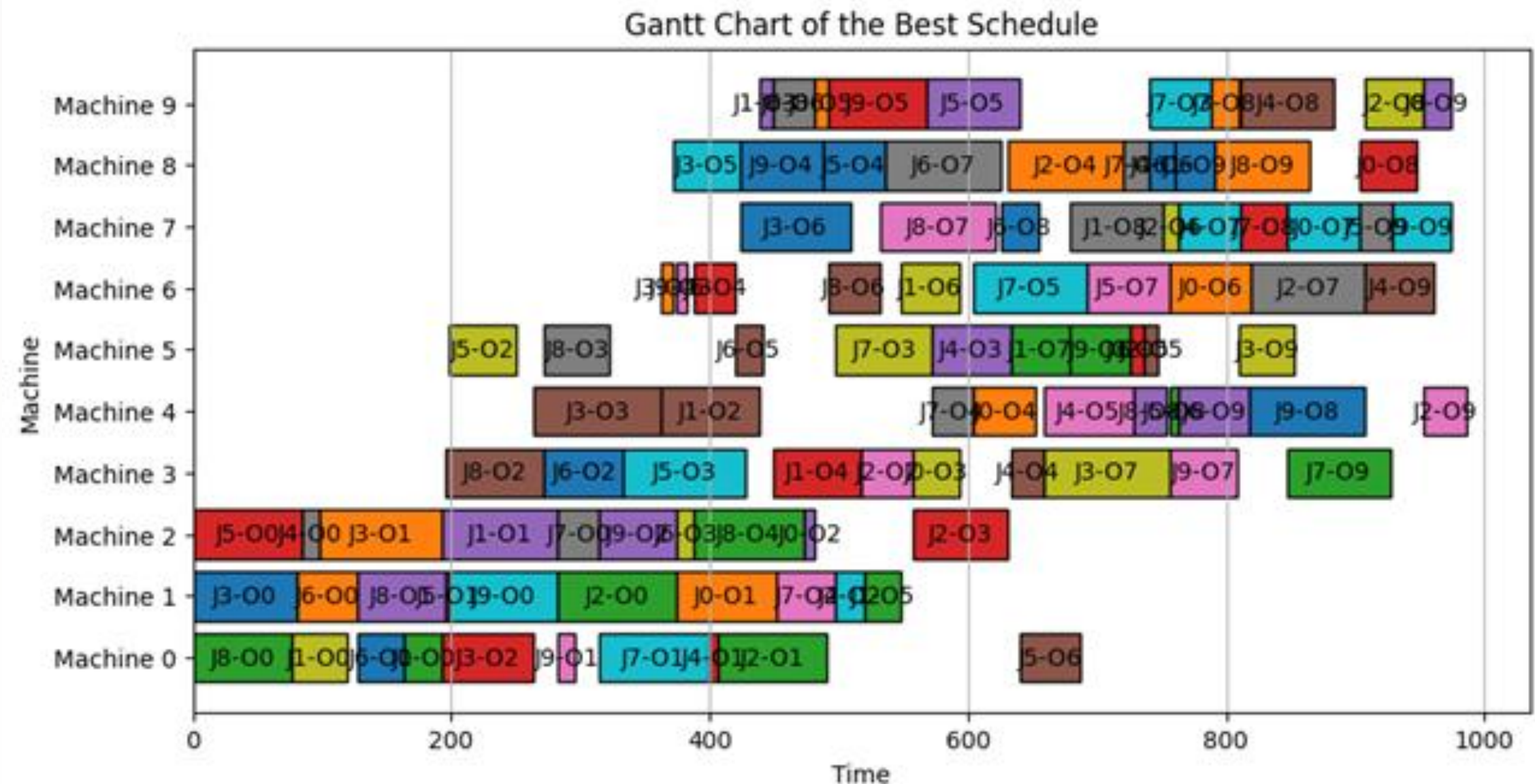
LA30 and LA35 reached the best-known solutions, while other instances produced high-quality near-optimal schedules.

Gantt Chart Visualization

Final Schedule Generated by Tabu Search

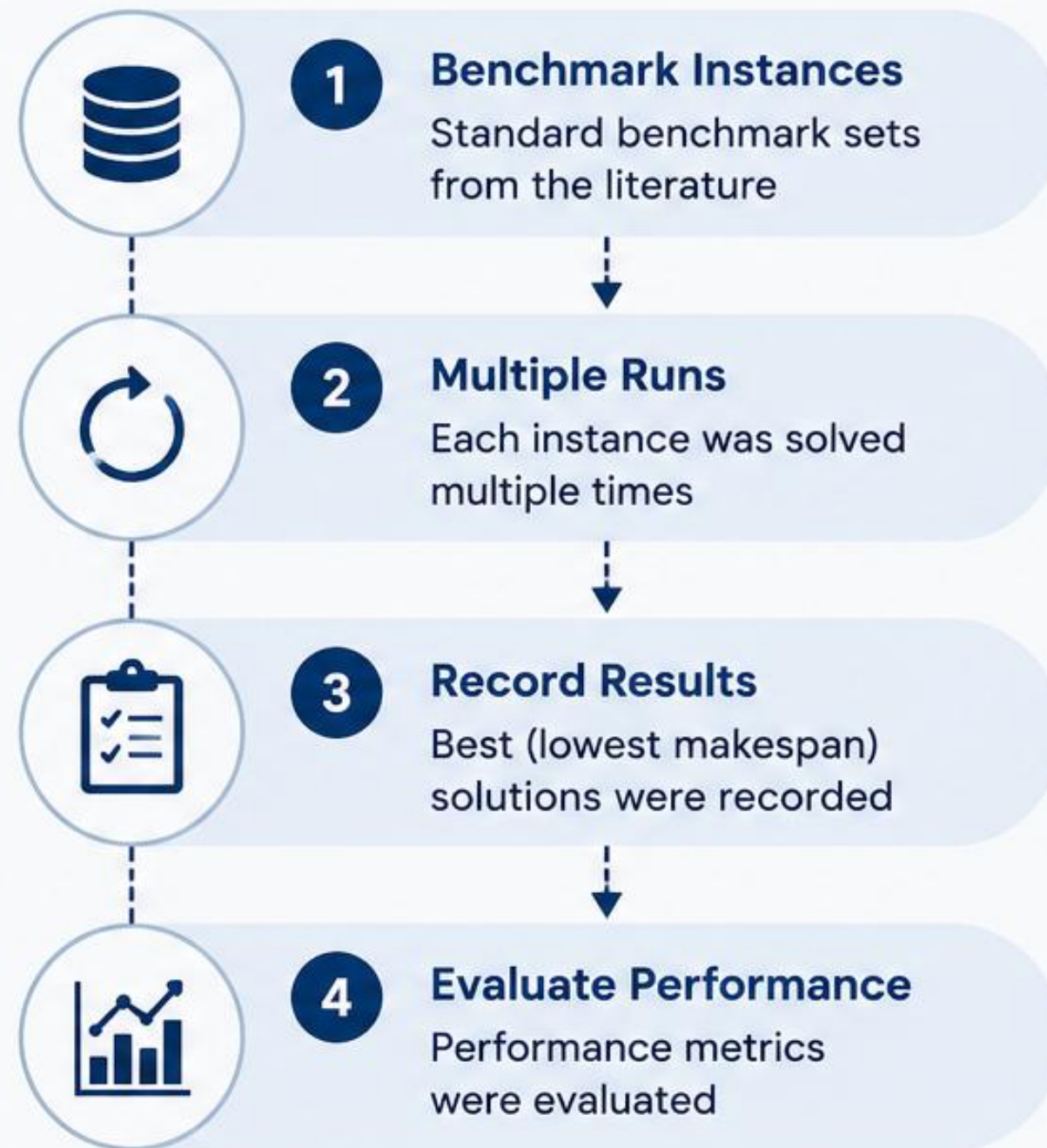
- Optimized machine schedule generated by Tabu Search
- Machine capacity conflicts are avoided
- Makespan (C_{max}) indicates total completion time

Example: Best FT10 schedule generated by Tabu Search



Experimental Procedure & Evaluation Metrics

EXPERIMENTAL PROCEDURE



EVALUATION METRICS



Makespan

The total completion time of the schedule



Computational Time

Total CPU time required to obtain the solution



Standard Deviation

Measures the variability of results across runs



Optimality Gap

Relative difference between best found and best known

Discussion and Future Improvements

Analysis of Algorithm Performance

-  Swap-based neighbourhood moves may limit exploration diversity
-  Larger benchmark instances showed higher variability
-  More advanced neighbourhood operators may improve solution quality
-  Computational time remained reasonable for all benchmark sets
-  Tabu Search successfully escaped local optima using memory structures

Conclusion



Main Findings

- Tabu Search successfully solved JSSP benchmark instances
- High-quality schedules were generated
- Effective convergence behavior was achieved
- Near-optimal solutions were obtained within reasonable computational time.



Future Work

- Hybrid metaheuristics
- Adaptive parameter tuning
- Multi-objective scheduling approaches

Thank You!